

PROMTS Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto: Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expression.

Objetivo: Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar a cabo la conversión. Instrucciones importantes: 1. No generes un proyecto nuevo desde cero. 2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo. 3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto. 4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse. 5. La solución debe estar hecha con programación orientada a objetos. 6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario. 7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz. 8. Se asume que la expression infija capturada es válida ya que de otra forma la conversión puede fallar. 9. Usa operadores básicos: +, -, *, /, \$(Potencia) y paréntesis. 10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversión, para esto genera un archivo .md que contenga esta explicación. Código existente:

```
from estructuras.lineales.lista_enlazada_simple import
LinkedList class OperacionesPila: def __init__(self, lista: LinkedList): self.lista = lista self.resultado = "" def
pila_agregar(self, data): self.lista.insert_at_beginning(data) self.resultado = f"Elemento {data} agregado
a la pila." return self.resultado def pila_eliminar(self): eliminado = self.lista.delete_first() if eliminado is
None: self.resultado = "La pila está vacía, no se puede eliminar." else: self.resultado = f"Elemento
{eliminado} eliminado de la pila." return eliminado def pila_tope(self): if self.lista.head is None:
self.resultado = "La pila está vacía." return None self.resultado = f"El tope de la pila es:
{self.lista.head.data}" return self.lista.head.data def pila_imprimir(self): if self.lista.head is None:
self.resultado = "La pila está vacía." return self.resultado elementos = [] temp = self.lista.head while
temp is not None: elementos.append(str(temp.data)) temp = temp.next self.resultado = " ->
".join(elementos) return self.resultado def is_empty(self): return self.top is None import sys from
PyQt5.QtWidgets import QApplication from load.load_MenuPrincipal import MenuPrincipal def main():
app = QApplication(sys.argv) ventana = MenuPrincipal() ventana.show() sys.exit(app.exec_()) if
__name__ == "__main__": main() from PyQt5.QtWidgets import QDialog from PyQt5 import uic from
menus.menu_pila import OperacionesPila from estructuras.lineales.lista_enlazada_simple import
LinkedList class LoadInterfazPila(QDialog): def __init__(self): super().__init__() self.ui =
uic.loadUi('ui/interfaz_pila.ui', self) # Crear la lista y el controlador de operaciones self.lista = LinkedList()
self.operaciones = OperacionesPila(self.lista) # Conectar botones
self.btn_pop.clicked.connect(self.pila_eliminar) self.btn_print.clicked.connect(self.pila_print)
self.btn_push.clicked.connect(self.pila_agregar) self.btn_top.clicked.connect(self.pila_tope) def
pila_agregar(self): dato = self.txt_dato.text() if dato.strip() == "": self.lbl_resultado.setText("Ingrese un
dato válido.") return self.operaciones.pila_agregar(dato)
self.lbl_resultado.setText(self.operaciones.resultado) def pila_eliminar(self): eliminado =
self.operaciones.pila_eliminar() self.lbl_resultado.setText(self.operaciones.resultado) def
pila_print(self): resultado = self.operaciones.pila_imprimir() self.lbl_resultado.setText(resultado) def
pila_tope(self): tope = self.operaciones.pila_tope() self.lbl_resultado.setText(self.operaciones.resultado)
```

```

from PyQt5.QtWidgets import QDialog

from PyQt5 import uic

from estructuras.lineales.nodo import Node

from estructuras.lineales.conversion import ConvertidorExpresiones


class LoadInterfazPila(QDialog):

    def __init__(self):
        super().__init__()

        self.ui = uic.loadUi('ui/untitled.ui', self)

        # Ya no usamos LinkedList, solo la pila
        self.operaciones = Node([])

        self.convertidor = ConvertidorExpresiones()

        # Conectar botón de conversión
        self.btn_convertir.clicked.connect(self.convertir_infija_a_posfija)

    def convertir_infija_a_posfija(self):
        expresion_infija = self.txt_expreccion.text()

        if not expresion_infija.strip():
            self.lbl_resultado.setText("Ingrese una expresión válida.")
            return

        try:
            resultado_posfija = self.convertidor.infija_a_posfija(expresion_infija)

            self.lbl_resultado.setText(resultado_posfija)

        except Exception as e:
            self.lbl_resultado.setText(f"Error: {str(e)}")

```

```
from estructuras.lineales.stack import Node
```

```
class ConvertidorExpresiones:
```

```
    def __init__(self):
```

```
        from estructuras.lineales.lista_enlazada_simple import LinkedList
```

```
        self.pila = Node (LinkedList())
```

```
    def prioridad(self, operador):
```

```
        if operador == '$': # Potencia
```

```
            return 3
```

```
        if operador in ('*', '/')
```

```
            return 2
```

```
        if operador in ('+', '-'):
```

```
            return 1
```

```
        return 0
```

```
    def infija_a_posfija(self, expresion):
```

```
        salida = []
```

```
        for token in expresion:
```

```
            if token.isalnum():
```

```
                salida.append(token)
```

```
            elif token == '(':
```

```
                self.pila.pila_agregar(token)
```

```
            elif token == ')':
```

```
                while not self.pila.is_empty() and self.pila.pila_tope() != '(':
```

```
                    salida.append(self.pila.pila_eliminar())
```

```
                self.pila.pila_eliminar()
```

```
            else:
```

```
                while (not self.pila.is_empty() and
```

```
        self.prioridad(self.pila.pila_tope()) >= self.prioridad(token)):
    salida.append(self.pila.pila_eliminar())
self.pila.pila_agregar(token)

while not self.pila.is_empty():
    salida.append(self.pila.pila_eliminar())

return " ".join(salida)
```

#PDF, EXPLICACION, PROMTS, EVIDENCIA